SAECOIN — GENESIS BLOCK SPECIFICATION

Document: Genesis_Block_Spec.md
Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Status: Draft / Planning
Consensus: Proof of Work
Transaction Model: UTXO

---

1. PURPOSE

This document defines the requirements for designing, generating, validating, and distributing the SAECOIN Genesis Block.

The Genesis Block is the first block in the SAECOIN blockchain.

It establishes the initial state of the network and provides the starting point for every full node.

Every compatible SAECOIN node must recognize the same valid Genesis Block for its network.

---

2. GENESIS BLOCK DEFINITION

The Genesis Block is:

- The first block of the blockchain.
- The parentless starting block.
- The initial blockchain reference.
- The starting point for block-height calculation.
- The foundation of the network's consensus history.

Genesis Block Height:

0

The first regular block will follow the Genesis Block.

Its height will be:

1

---

3. NETWORK SEPARATION

SAECOIN will have separate Genesis Blocks for:

1. Development Network
2. Testnet
3. Mainnet

Each Genesis Block must have a different identity.

Property| Development| Testnet| Mainnet
Genesis block| Unique| Unique| Unique
Genesis hash| Unique| Unique| Unique
Network ID| Separate| Separate| Separate
Chain state| Separate| Separate| Separate
Wallet network| Separate| Separate| Separate

A node must reject a Genesis Block that does not match its configured network.

---

4. GENESIS BLOCK STRUCTURE

The final binary structure will be determined by the Blockchain Architecture and Consensus Specifications.

The proposed logical structure is:

GenesisBlock
  Header
    Version
    PreviousBlockHash
    MerkleRoot
    Timestamp
    DifficultyTarget
    Nonce

  Transactions
    GenesisTransaction

The exact field sizes, byte order, serialization rules, and hash algorithm remain TO BE FINALIZED.

Genesis serialization must be deterministic.

---

5. GENESIS BLOCK HEADER

The header should contain:

Field| Description| Status
Version| Block format version| Provisional
Previous block hash| Parent block reference| Genesis-specific
Merkle root| Transaction commitment| Provisional
Timestamp| Genesis creation timestamp| Not Finalized
Difficulty target| Initial PoW target| Not Finalized
Nonce| PoW solution value| Not Finalized

For the Genesis Block:

The previous block hash must follow a precisely defined Genesis-specific rule.

It must not depend on uninitialized or arbitrary memory.

A canonical zero value may be used if selected by the final protocol.

---

6. GENESIS TRANSACTION

The Genesis Block may contain a special genesis transaction.

The final protocol must define:

- Whether the Genesis Block contains a coinbase-like transaction.
- Whether the transaction creates spendable outputs.
- Whether any initial allocation exists.
- Whether the Genesis Block creates zero spendable coins.
- How the initial UTXO state is represented.

Proposed policy:

Premine: 0 SAC
Initial allocation: 0 SAC

No founder, developer, company, or organization allocation should be created unless the project explicitly changes this policy through a documented decision.

The Genesis Block must not accidentally create spendable coins outside the verified monetary issuance schedule.

---

7. GENESIS TIMESTAMP

The Genesis Block timestamp must be selected and documented.

Requirements:

- Use a fixed timestamp.
- Store it in the agreed protocol format.
- Use the same timestamp in every compatible implementation.
- Do not generate a new timestamp automatically during normal node startup.

The final timestamp remains TO BE DECIDED.

The timestamp must be selected before the Genesis Block hash is calculated.

---

8. INITIAL DIFFICULTY

The Genesis Block must define the initial Proof of Work difficulty target.

The following details must be finalized:

- PoW algorithm
- Hash function
- Target representation
- Initial difficulty
- Compact target encoding, if used
- Validity rules
- Genesis-specific validation behavior

The initial target must be valid under the finalized consensus rules.

The Genesis Block must not bypass normal security requirements through an undocumented exception.

---

9. GENESIS BLOCK HASH

The Genesis Block hash identifies the exact Genesis Block.

It must be calculated using the finalized header serialization and hash algorithm.

The final implementation must guarantee:

- Deterministic serialization
- Deterministic hash calculation
- Identical results across compatible nodes

- Correct byte order
- No platform-dependent behavior

The final Genesis Block hash will be recorded in the network configuration.

No developer or node operator should be able to generate a different mainnet Genesis Block accidentally.

---

10. GENESIS BLOCK GENERATION PROCESS

The Genesis Block generation process should follow these steps:

1. Finalize the consensus specification.
2. Finalize the block header format.
3. Finalize transaction serialization.
4. Finalize the PoW algorithm.
5. Select the Genesis timestamp.
6. Select the initial difficulty target.
7. Construct the Genesis transaction, if required.
8. Calculate the transaction identifier.
9. Calculate the Merkle root.
10. Construct the block header.
11. Search for a valid nonce if Genesis PoW requires it.
12. Calculate the Genesis Block hash.
13. Independently verify the result.
14. Record all final Genesis parameters.
15. Add the verified values to the network configuration.

The generation process must be reproducible.

---

11. GENESIS BLOCK VALIDATION

Every node must validate its configured Genesis Block.

Validation should verify:

- Correct network identity
- Correct Genesis hash
- Correct header structure
- Correct transaction structure
- Correct Merkle root
- Correct timestamp
- Correct difficulty target
- Correct Proof of Work, if required

- Correct initial monetary state

Genesis validation should not depend on external servers.

The node must be able to verify the Genesis Block locally.

---

12. GENESIS CONFIGURATION

The final network configuration should contain a fixed Genesis reference.

Proposed configuration structure:

```
[network]
name = "SAECOIN"
network_id = "TO_BE_FINALIZED"

[genesis]
height = 0
timestamp = "TO_BE_FINALIZED"
hash = "TO_BE_FINALIZED"
merkle_root = "TO_BE_FINALIZED"
nonce = "TO_BE_FINALIZED"
difficulty_target = "TO_BE_FINALIZED"
```

This is an illustrative configuration.

The actual format and fields must match the final implementation.

Do not use placeholder values in a production mainnet configuration.

---

13. CROSS-PLATFORM VERIFICATION

The Genesis Block must produce identical results on supported platforms.

Verification should include:

- Windows 11
- Linux
- Android, where relevant to wallet verification
- Different CPU architectures, if supported

The following must remain identical:

- Serialized block header

- Genesis transaction serialization
- Merkle root
- Genesis Block hash

Platform-dependent serialization is not permitted.

---

14. GENESIS BLOCK TESTING

Required tests:

14.1 Deterministic Generation Test

Repeated generation with identical parameters must produce identical results.

14.2 Hash Verification Test

The generated Genesis Block hash must match the published reference value.

14.3 Network Separation Test

A mainnet node must reject testnet and development Genesis Blocks.

14.4 Invalid Genesis Test

Modified Genesis fields must fail validation.

14.5 Merkle Root Test

The Merkle root must match the expected transaction set.

14.6 Cross-Platform Test

All supported implementations must calculate the same Genesis Block hash.

14.7 Initial Supply Test

The Genesis Block must not violate the approved monetary issuance policy.

---

15. GENESIS MANIFEST

Before mainnet launch, a permanent Genesis Manifest should be published.

It should include:

Item| Final Value
Project name| SAECOIN
Ticker| SAC (Provisional)
Network ID| Not Finalized
Genesis timestamp| Not Finalized
Genesis hash| Not Finalized
Merkle root| Not Finalized
Nonce| Not Finalized
Initial difficulty| Not Finalized
PoW algorithm| Not Finalized
Hash function| Not Finalized
Initial spendable supply| 0 SAC proposed

The manifest should also document:

- Serialization version
- Genesis generation software version
- Source code commit
- Verification procedure
- Independent verification results

---

16. GENESIS BLOCK SECURITY RULES

1. Never change the mainnet Genesis Block silently.
2. Never generate a new mainnet Genesis timestamp during startup.
3. Never trust a remote peer's Genesis Block without verification.
4. Never allow development configuration to overwrite mainnet configuration.
5. Never introduce undocumented initial allocations.
6. Never modify the Genesis hash to hide a consensus error.
7. Verify all Genesis parameters independently.
8. Preserve the final Genesis Manifest permanently.

Changing the Genesis Block after network launch would effectively create a different chain identity and must be treated as a major protocol decision.

---

17. MAINNET GENESIS RELEASE CHECKLIST

Before mainnet launch:

- [ ] Consensus specification finalized.
- [ ] PoW algorithm finalized.
- [ ] Hash function finalized.
- [ ] Block serialization finalized.
- [ ] Transaction serialization finalized.

- [ ] Initial difficulty finalized.
- [ ] Timestamp finalized.
- [ ] Genesis transaction policy finalized.
- [ ] Premine policy verified.
- [ ] Genesis Block generated.
- [ ] Genesis hash independently verified.
- [ ] Merkle root independently verified.
- [ ] Cross-platform tests passed.
- [ ] Network identity finalized.
- [ ] Genesis Manifest published.
- [ ] Mainnet configuration frozen.

---

18. FINAL PRINCIPLES

The Genesis Block must be:

- Deterministic
- Reproducible
- Independently verifiable
- Identical across compatible mainnet nodes
- Consistent with monetary rules
- Consistent with consensus rules
- Permanently documented

The Genesis Block is not merely a configuration file.

It is a foundational part of SAECOIN's network identity.

DOCUMENT STATUS

Document: Genesis_Block_Spec.md
Version: 1.0
Project: SAECOIN
Genesis Height: 0
Premine: 0 SAC proposed
Genesis Hash: Not Generated
Genesis Timestamp: Not Finalized
PoW Algorithm: Not Finalized
Mainnet: Not Launched
Implementation Status: Not Started